



STORAGE PERSISTENCE FOR PODS

Book 05 of the series — giving data a life of its own: Volumes, the PV/PVC divorce of supply and demand, access modes decoded (RWO $\neq$ one Pod!), reclaim policies, StorageClasses & dynamic provisioning, StatefulSet per-replica storage, and the CSI universal plug.



Volumes
pod attachments



PV
cluster supply



PVC
the request



RWO·ROX·RWX
decoded



Reclaim
Retain · Delete



StorageClass
build on demand



StatefulSet
storage per replica



CSI
universal plug

PART I · THE PERSISTENCE PROBLEM

- ☐ **CH 01** Why Storage Is Different p.3
- ☐ **CH 02** Volumes — The Pod's Attachments p.4

PART II · PV & PVC

- ☐ **CH 03** PV & PVC — Supply Meets Demand p.5
- ☐ **CH 04** The Binding Process p.6
- ☐ **CH 05** Access Modes — RWO·ROX·RWX Decoded .. p.7
- ☐ **CH 06** Reclaim Policies & the PV Lifecycle p.8

PART III · DYNAMIC PROVISIONING

- ☐ **CH 07** StorageClasses — Build on Demand p.9
- ☐ **CH 07-b** StorageClasses (cont.) p.10
- ☐ **CH 08** Class Tuning — Binding Modes & Expansion p.11

PART IV · STATEFUL STORAGE

- ☐ **CH 09** StatefulSets — Why Identity Matters p.12
- ☐ **CH 10** volumeClaimTemplates — Storage Per Replica p.13

PART V · UNDER THE HOOD

- ☐ **CH 11** CSI — The Universal Plug p.14

PART VI · LABS

- ☐ **LAB 1** Manual PV ↔ PVC Binding p.15
- ☐ **LAB 2** Dynamic Provisioning + Reclaim Proof p.16
- ☐ **LAB 3** StatefulSet — Identity & Per-Replica Data .. p.17
- ☐ **LAB 3-b** StatefulSet (cont.) p.18
- ☐ **LAB 4** The Data-Survival Gauntlet p.19

PART VII · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.20
- ☐ **KEY** Answer Key + Scoring p.21
- ☐ **GLOSS** Glossary p.22
- ☐ **CHEAT** One-Page Cheat Sheet p.23
- ☐ **FINAL** Where to Next → Book 06 p.24



Assumes Books 01–04. Book 01 · CH 11 already told you PVs are **cluster-scoped** — now you'll see why. Book 04 · CH 10's `--delete-emptydir-data` flag was this book knocking: *emptyDir dies with the Pod, and drain makes you choose*. Book 02's Secrets return as CSI credentials (CH 11).

Everything so far assumed Pods are expendable — and their **filesystem is too**. A container's writable layer is built fresh from the image every time the container is replaced. Data written "inside the container" has a death wish.

THE THREE LIVES OF DATA — HOW LONG DOES IT SURVIVE?

- 1 • **Container writable layer** dies with the CONTAINER
crash → kubelet restarts → fresh layer → your writes GONE
- 2 • **emptyDir volume** dies with the POD
survives container restarts — deleted when the Pod leaves the node
- 3 • **PersistentVolumes** outlives EVERYTHING
pods, nodes, even the cluster's opinion — data remains

✗ FINE WITHOUT PERSISTENCE

Stateless APIs (state lives in a DB *elsewhere*)

Static sites, proxies, workers

Scratch space, temp files (emptyDir!)

💀 DIES WITHOUT PERSISTENCE

Databases (MySQL, Postgres — Book 01's YAML wrote nothing durable!)

User uploads, sessions

Message queues, ML models, indexes



Book 04 foreshadowed this: `drain --delete-emptydir-data` exists precisely because emptyDir data dies with the Pod — drain makes you *admit* it. And Book 01's MySQL lab ran on the container layer: that database was one crash away from amnesia.



One sentence: Kubernetes treats Pods as cattle — so anything that must **outlive a Pod** needs to live *outside* the container layer, declared as a Volume at minimum and a PersistentVolume at best. This book is the ladder from scratch space to real persistence.

▶▶ Step one of the ladder — [CH 02: Volumes — The Pod's Attachments](#)

↑ back to top



A Volume is a **directory, declared at Pod level, mounted into containers**. It outlives the containers in the Pod — but its lifespan depends on its **type**.

```
spec:
  containers:
  - name: app
    image: myapp:2.0
    volumeMounts:
    - { name: scratch, mountPath: /tmp/cache }
  volumes:
    # declared at POD level — shared by containers
  - name: scratch
    emptyDir: {}
```

Type	Backing	Lifespan	Use for
<code>emptyDir</code>	node disk / RAM (<code>medium: Memory</code>)	Pod's life	scratch, multi-container share, checkpointing
<code>hostPath</code>	a path on the node	survives the Pod — tied to the node	⚠ node agents, privileged daemons only
<code>configMap</code> / <code>secret</code>	API objects (Book 02 · CH 02/06)	Pod's life, kubelet-synced	config & credentials as files
<code>nfs</code>	an NFS share	independent of Pod	shared file storage
<code>persistentVolumeClaim</code>	a PV via binding	independent — the star ★	real persistence (Part II)
<code>csi</code>	any CSI driver (CH 11)	driver-defined	ephemeral CSI volumes



hostPath is a loaded weapon: it hard-couples your Pod to *a specific path on whatever node it lands on* — rescheduling to another node = "where did my data go?" It can also read **any path on the node** (including `/`) — a container-escape classic. Legit uses: kubelet-adjacent daemons (log shippers, CNI). Everything else: use a PVC.



Shared fate, shared files:

two containers in one Pod mounting the same Volume = the Book 01 · CH 19 sidecar pattern's backbone (the log-shipper read the app's file from one)



`emptyDir.sizeLimit` caps growth — beyond it, eviction (kubelet treats it like disk pressure)



One sentence: a Volume is a **Pod-scoped directory** whose *backing* (and lifespan) is chosen by its type — from RAM-speed scratch (`emptyDir`) to node-coupled files (`hostPath`) to the fully independent PVC, which is where the rest of this book lives.



The star of the ladder — [CH 03: PV & PVC — Supply Meets Demand](#)

↑ back to top



The genius of Kubernetes storage is a **divorce of concerns**: the people who *provision* storage (ops, storage admins) and the people who *consume* it (developers) never talk to each other. They meet through two objects.

PERSISTENTVOLUME — THE SUPPLY

```
apiVersion: v1
kind: PersistentVolume
metadata: { name: pv-warehouse-1 }
spec:
  capacity:
    storage: 5Gi
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  hostPath: { path: /data/pv1 }
```

Cluster-scoped (Book 01 · CH 11 promised!).
Real storage: NFS, cloud disk, hostPath. No namespace, owned by admins.

PERSISTENTVOLUMECLAIM — THE DEMAND

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: db-data, namespace: shop }
spec:
  resources:
    requests:
      storage: 3Gi
  accessModes: [ReadWriteOnce]
  storageClassName: "" # "" = manual binding only (CH 07)
```

Namespaced. Not storage — a *wish*: "I need 3Gi, RWO." The developer's entire vocabulary of storage.

THE MEETING — BINDING TURNS TWO WISHES INTO A CONTRACT



PVC bound → Pods mount the *claim*, never the volume: `volumes: [{name: data, persistentVolumeClaim: {claimName: db-data}}]`



The apartment analogy, upgraded: the PV is the **warehouse in the building's basement** (building-wide, managed by the landlord). The PVC is your **lease application** ("3Gi storage, I need to write to it"). You never touch the warehouse — you mount your *lease*.



One sentence: *PV = cluster-scoped real storage (supply) · PVC = namespaced request (demand) · binding = the matchmaking* — and Pods only ever reference the PVC, keeping consumers and infrastructure permanently decoupled.

How the matchmaker works — [CH 04: The Binding Process](#)

↑ back to top



A controller (the **PV controller**, inside kube-controller-manager) continuously matches Pending claims to Available volumes. Four checks, all must pass:

THE MATCHER'S FOUR QUESTIONS

1 · **Capacity** PV $\geq$ request?

2 · **Access modes** PV $\supseteq$ requested?

3 · **storageClassName** exact match?

4 · **selector** labels match?

smallest PV that fits wins · binding is 1:1 — a bound PV is spoken for

Check	Rule	Gotcha
Capacity	PV's <code>storage</code> $\geq$ PVC's request	you get the <i>whole</i> PV, not a slice — a 3Gi claim on a 5Gi PV wastes 2Gi
Access modes	PV offers <i>at least</i> what the claim asks	claim <code>[RWO]</code> can bind a PV offering <code>[RWO, RWX]</code>
StorageClass	exact string match — including <code>""</code>	claim with <code>className: ""</code> binds <i>only</i> PVs also stamped <code>""</code> ; a claim with <i>no</i> field uses the default class (CH 07)
Selector	claim's <code>selector.matchLabels</code> $\subseteq$ PV labels	ops tags PVs (<code>tier: gold</code>), devs target the tag

```
$ kubectl get pvc db-data
NAME      STATUS    VOLUME      CAPACITY  ACCESS MODES
db-data   Pending                                     # nobody fits — check the 4 questions
db-data   Bound     pv-warehouse-1  5Gi       RWO
```

- Pre-binding:** a PV can name its soulmate `spec.claimRef` — skips the matchmaking entirely (migrations, DR drills)
- Binding is exclusive:** one PV $\leftrightarrow$ one PVC, period. Sharing storage between Pods happens *through the same claim*, not through multiple bindings



Pending PVC triage: 90% of stuck claims are the **storageClassName mismatch** — claim says `""` but only classed PVs exist, or vice versa. The other 10%: capacity too small, access-mode subset missing, or selector pointing at labels nobody has.



One sentence: binding = capacity $\geq$, modes $\supseteq$, class ==, labels match — *smallest fitting PV wins, exactly one claim per PV*. Every Pending PVC is one of those four questions answered wrong.

The matching criterion with the most lies attached — [CH 05: Access Modes Decoded](#)

[↑ back to top](#)

The names are honest about read/write but **lying about the unit**. The unit is **NODES**, not Pods — and that one fact explains half of all storage tickets.

Mode	Name says	Really means	Typical backing
<code>ReadWriteOnce</code>	one writer	read-write mounted by ONE NODE	cloud block disks: EBS, PD, Azure Disk
<code>ReadOnlyMany</code>	many readers	read-only on MANY nodes	NFS, EFS (ro), pre-staged datasets
<code>ReadWriteMany</code>	shared write	read-write on MANY nodes	NFS, EFS, Azure Files, CephFS
<code>ReadWriteOncePod</code> newer	one writer	read-write by ONE POD — truly exclusive (GA 1.29)	block disks, with the strictest semantics

 **The interview trap: RWO ≠ one Pod.** It's one *node* — so **three Pods on the same node can all mount one RWO volume simultaneously**, while the same three Pods spread across three nodes cannot. When your scaled Deployment mysteriously wedges, this is why: replicas scattered across nodes fight over one RWO claim.

MULTI-ATTACH — THE CLASSIC FAILURE

```
$ kubectl describe pod db-2 | tail -2
Events:
  Multi-Attach error for volume "pvc-abc":
  Volume is already exclusively attached to one node
  and can't be attached to another
# Pod B (other node) wants the RWO volume Pod A's node holds.
# The volume driver physically can't attach a block disk twice.
```

-  **Block storage = RWO-only:** a cloud disk plugs into one machine — RWX demands a *shared filesystem* (NFS/EFS/Ceph) or an in-cluster one (NFS provisioner, Longhorn, EFS CSI)
-  **Check reality, not hopes:** `kubectl get pv` shows the PV's ACCESS MODES — a claim asking RWX will sit **Pending forever** against an RWO-only fleet

 **One sentence:** access modes are promises about **node-level attach** — RWO = one node (not one Pod!), RWX = needs genuinely shared backing — and `Multi-Attach error` in events is the sound of a block disk refusing to be in two places at once.

 What happens when the love ends — [CH 06: Reclaim Policies](#)

↑ back to top



A PV has a whole life *after* its claim dies. The **reclaim policy** decides what that afterlife looks like — and whether your data survives it.

Available → Bound → PVC deleted → Released → policy decides →

Retain: stays **Released** forever — data intact, ADMIN must intervene

Delete: PV object + **backing storage** both destroyed ← dynamic default

Recycle: scrub & re-offer — **REMOVED** from the API (dynamic provisioning replaced it)

Policy	PVC deleted →	Data	Use when
Retain	PV → Released , never auto-reused	kept — hostage until an admin acts	databases, anything you'd cry over
Delete	PV + cloud disk annihilated	gone	truly ephemeral state, CI scratch

```
# the Released hostage situation — and the escape:
$ kubectl get pv
NAME      CAPACITY  CLAIM              RECLAIM  STATUS
pv-db     5Gi       shop/db-data       Retain   Released
# reuse it manually: clear the claimRef → back to Available
# (data intact! next binding re-attaches the same disk)
$ kubectl patch pv pv-db -p '{"spec":{"claimRef": null}}'
```



Two honest warnings: (1) the **pvc-protection finalizer** — a PVC in use by a Pod shows **Terminating** indefinitely; Kubernetes refuses to yank storage from a running Pod. (2) **Default is Delete for dynamic volumes** — an innocent `kubectl delete pvc` on a Delete-class volume is a production data-loss event. Patch the PV (or the class, CH 08) to **Retain** for anything real.

Released ≠ Available: Released PVs are NOT re-matchable — that's the point of Retain (no accidental rebinding of dirty state)

Audit trick: `kubectl get pv -o custom-columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy,STATUS:.status.phase` — one glance, whole estate



One sentence: the PV lifecycle is *Available → Bound → Released → {Retain: wait for admin | Delete: erase everything}* — and the difference between "I deleted a claim" and "I deleted the company's data" is one policy field.

What if you never write a PV at all? — [CH 07: StorageClasses](#)

↑ back to top



Writing PVs by hand doesn't scale — it's the landlord hand-building a warehouse per lease. A **StorageClass** is the **construction company**: PVCs name it, and it builds the warehouse automatically.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
  annotations:
    storageclass.kubernetes.io/is-default-class: "true" # THE default
provisioner: ebs.csi.aws.com # WHO builds (CSI driver – CH 11)
parameters: # the build spec, vendor-specific
  type: gp3
  encrypted: "true"
reclaimPolicy: Delete # default for dynamic volumes
allowVolumeExpansion: true # grow later (CH 08)
volumeBindingMode: WaitForFirstConsumer # smart timing (CH 08)
```

DYNAMIC PROVISIONING — THE WHOLE DANCE, AUTOMATED



zero PVs hand-written · the SC name in the claim is the only thing a developer must know

PVC's storageClassName	Result
names a class (<code>fast</code>)	dynamic provisioning from that class — manual PVs of the same name are bypassed unless matching perfectly
<i>field omitted</i> + a default class exists	uses the default class — <code>standard</code> on minikube/kind
<i>field omitted</i> + no default	old-school manual binding only — usually Pending forever
<code>""</code> (empty string)	explicitly opts OUT of dynamic — manual PVs only (CH 04's matcher)



Classes are also a menu: `fast` (NVMe), `cheap` (HDD), `shared` (NFS/RWX) — the same cluster offers different storage *tiers*, and the developer picks by writing one name. Multi-cloud? One class per backend, one field per claim.



One sentence: a StorageClass is a **named recipe** — **provisioner** + **parameters** + **policy** — **that builds PVs on demand**; claims name the recipe, the default class catches the lazy, and `""` is the explicit "I brought my own warehouse."

▶ The two settings that separate toy classes from production ones — [CH 08: Class Tuning](#)

↑ back to top

 **volumeBindingMode — WHEN to bind****IMMEDIATE (DEFAULT)**

Bind the instant the PVC exists — **before any Pod exists**. Simple, but with zoned storage the disk may be born in **zone A** while your **Pod later schedules to zone C** → attach impossible → Pod wedged.

```
0/1 nodes available:
volume node affinity conflict # ← this error = bound to the wrong zone
```

WAITFORFIRSTCONSUMER

The PVC waits in **Pending** until a **Pod that uses it is scheduled** — then provisions the disk **in the Pod's zone**. Topology-correct by construction. Production default for zoned clouds.

```
# PVC waits... pod schedules to zone-c...
# disk born IN zone-c — attach just works
```

allowVolumeExpansion — grow, never shrink

```
# class must allow it (CH 07)... then just edit the claim:
$ kubectl patch pvc db-data -p '{"spec":{"resources":{"requests":{"storage":"10Gi"}}}}'
$ kubectl get pvc db-data
NAME      CAPACITY  STATUS
db-data   10Gi      Bound    # filesystem resized online (most drivers)
# SHRINKING IS IMPOSSIBLE — storage can only ever grow. plan headroom.
```

Setting	Recommended	Why
volumeBindingMode	WaitForFirstConsumer (zoned clouds)	disk and Pod land in the same zone, always
allowVolumeExpansion	true	DBs outgrow their bids; growing online beats migrating
reclaimPolicy	Retain for real data classes	CH 06: default Delete is a loaded foot-gun
parameters	vendor-specific (type: gp3 ...)	the tier definition itself



Debugging zoned wedges: the PV inherits the zone as a **nodeAffinity** (`kubectl describe pv` → `Node Affinity` section) — the Scheduler literally cannot place Pods elsewhere. Wrong-zone = delete PVC + Pod, recreate with `WaitForFirstConsumer`. (Topologies from Book 04 · CH 08 pay off here.)



One sentence: production classes set **WaitForFirstConsumer** (zone-correct disks), **allowVolumeExpansion: true** (grow-only flexibility), and **Retain** for anything with a pulse — Immediate binding on zoned storage is how disks get born in the wrong ZIP code.

Now the workload that truly needs all this — [CH 09: StatefulSets](#)

↑ back to top



A Deployment's Pods are **interchangeable cattle** — random names, any replica can serve any request. Databases are **not cattle**: replica 0 is the primary, replica 1 is its follower, and each owns *its own* data. Enter the StatefulSet.

DEPLOYMENT IDENTITY

```
web-7d9f-x7k2p  random
web-7d9f-bbbba  random
die → replaced by NEW random names
any pod can serve anything
no "which one owns the data?"
```

STATEFULSET IDENTITY

```
db-0  PRIMARY — born first
db-1  follower — waits for 0
db-2  follower — waits for 1
die → RESURRECTED WITH THE SAME NAME
+ same storage (CH 10) + same DNS
```

The headless service — per-Pod DNS

```
apiVersion: v1
kind: Service
metadata: { name: db }
spec:
  clusterIP: None          # HEADLESS — no VIP, no balancing
  selector: { app: db }
  ports: [{ port: 5432 }]
# instead of one VIP, DNS returns EACH POD's address:
# db-0.db.shop.svc.cluster.local → db-0's IP (the primary!)
# followers find the primary BY NAME — no IPs in any config
```

Ordered by default: 0 → Ready → then 1 → Ready → then 2; deletion runs in reverse (2 → 1 → 0). The bootstrap sequence a database needs, built in

Name = contract: replication configs say "follow `db-0`" — and it's true forever, across rescheduling and crashes



Scope note: this chapter covers identity + storage — the *reason* StatefulSets exist for data. The full lifecycle (rolling updates, `partition` canaries, OnDelete) is Book 06's territory.



One sentence: StatefulSets give each replica a **stable name, stable DNS, and ordered startup** — the preconditions for "replica 0 owns the data" to mean anything; next chapter attaches the storage that makes the promise complete.

The storage half of the contract — [CH 10: volumeClaimTemplates](#)

↑ back to top

volumeClaimTemplates — Storage Per Replica

The StatefulSet's killer storage feature: instead of one shared claim, you embed a **template** — and each replica gets its **own PVC stamped from it**, named after the Pod it belongs to.

```
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: db }
spec:
  serviceName: db                # the headless service (CH 09)
  replicas: 3
  selector: { matchLabels: { app: db } }
  template:                      # the pod stamp
    metadata: { labels: { app: db } }
    spec:
      containers:
        - name: db
          image: postgres:16
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
  volumeClaimTemplates:         # ★ THE COOKIE CUTTER FOR CLAIMS
    - metadata: { name: data }
      spec:
        accessModes: [ReadWriteOnce]
        resources: { requests: { storage: 10Gi } }
        storageClassName: fast
```

StatefulSet db (replicas: 3)

```
├─ pod db-0  ⇔ PVC data-db-0 ⇔ PV (own disk)
├─ pod db-1  ⇔ PVC data-db-1 ⇔ PV (own disk)
└─ pod db-2  ⇔ PVC data-db-2 ⇔ PV (own disk)
naming law: <template-name>-<statefulset>-<ordinal>
```

✓ SCALE UP → CLAIMS BORN

3 → 4 replicas: `data-db-3` dynamically provisioned (CH 07!), mounted by `db-3`. Fresh disk, zero setup.

⚠ SCALE DOWN → CLAIMS RETAINED

4 → 2: Pods `db-3`, `db-2` die — but **their PVCs stay** (data-safety by design). Scale back up and the same replicas **reunite with their old disks**.



Deleting the StatefulSet deletes NOTHING storage-wise — every `data-db-N` survives (Retain-like behavior, baked in). Cleanup is manual: `kubectl delete pvc data-db-0 data-db-1 ...` This is the storage counterpart of Book 01's cascade lesson — *some deletes cascade (namespaces), some deliberately don't (STS claims)*.



One sentence: `volumeClaimTemplates` = **one PVC per replica, named for its Pod, born on scale-up, kept on scale-down** — the mechanism that lets `db-0` mean "the primary *and its exact disk*" forever.

Who builds the disks behind it all — [CH 11: CSI](#)

↑ back to top



Every provisioner you've met (`ebs.csi.aws.com`, minikube's `hostpath`, Ceph...) speaks **CSI — the Container Storage Interface**: a vendor-neutral gRPC contract that lets *any* storage backend plug into *any* Kubernetes.

🔦 BEFORE CSI (IN-TREE ERA)

Driver code lived **inside the k8s source tree**

Vendors begged SIGs for inclusion; releases tied to k8s cycles

k8s shipped hundreds of drivers it couldn't all test

🔦 AFTER CSI

Drivers are **containers you install** (Helm charts)

Vendors release on **their** schedule, any k8s version

In-tree drivers: **frozen & deprecated** — CSI is the only growth path

Pod $\Rightarrow$ PVC $\Rightarrow$ `StorageClass.provisioner = "ebs.csi.aws.com"`

|

▼

CSI driver — two halves, both just Pods:

- └ `Controller plugin` (Deployment) — `CreateVolume`, `DeleteVolume`, `Expand`
- └ `Node plugin` (DaemonSet — one per node, Book 04 CH 10 skips these!)
 - └ `NodeStageVolume` / `NodePublishVolume` → the disk appears at the `mountPath`

🔑 **Credentials, of course, are Secrets** (Book 02 · CH 05–06): the driver's chart consumes a Secret holding cloud API keys / backend tokens

⚙️ **volumeMode:** `Filesystem` (default — a formatted dir) vs `Block` (raw device handed to the Pod — databases that manage their own formatting love it)

🌐 `ebs.csi.aws.com` · `pd.csi.storage.gke.io` · `disk.csi.azure.com` · `rbd.csi.ceph.com` · `driver.longhorn.`

Examples
in the
wild:



Reading a class like a pro: the `provisioner` field is a CSI driver identity. If a class says `kubernetes.io/aws-ebs` (old in-tree name) on a modern cluster — that's a legacy class on migration watch.



One sentence: CSI is the **USB standard of storage** — a stable plug spec that turned storage drivers from kernel-adjacent k8s features into ordinary installable containers, which is why every class you'll ever write ends in a `...csi...` provisioner.

▶▶ Prove all of it with your hands — [Part VI: Labs](#)

↑ back to top



STEP 1 · Provision supply by hand

```
$ kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolume
metadata: { name: pv-manual-1, labels: { tier: lab } }
spec:
  capacity: { storage: 1Gi }
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
  hostPath: { path: /data/pv-manual-1, type: DirectoryOrCreate }
EOF
$ kubectl get pv pv-manual-1 # STATUS: Available (2 of the 4 checks pre-passed)
```

STEP 2 · Demand + consume

```
$ kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: lab-claim }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 500Mi } }
  storageClassName: "" # match the PV's "" - CH 04 rule 3
  selector: { matchLabels: { tier: lab } }
EOF
$ kubectl get pvc lab-claim # Bound → pv-manual-1 (smallest fitting PV)
$ kubectl run writer --image=busybox:1.36 --restart=Never --rm -it \
  --overrides='{ "spec": { "containers": [ { "name": "w", "image": "busybox:1.36",
  "command": [ "sh", "-c", "echo persistent rocks > /data/hello.txt; sleep 5" ],
  "volumeMounts": [ { "name": "d", "mountPath": "/data" } ] },
  "volumes": [ { "name": "d", "persistentVolumeClaim": { "claimName": "lab-claim" } } ] } }'
```

00 PROVE IT'S THE WAREHOUSE

```
$ minikube ssh "cat /data/pv-manual-1/hello.txt"
persistent rocks
# the file lives on the NODE's disk,
# inside the PV's hostPath -
# not in any container
```

THE 4 CHECKS, VERIFIED

```
capacity: 1Gi ≥ 500Mi ✓
modes:    RWO ⊇ RWO ✓
class:    "" == "" ✓
selector: tier=lab ✓
→ smallest-fitting win, 1:1 bound
```

Teardown: `kubectl delete pvc lab-claim` → PV goes **Released** (Retain! CH 06) → `kubectl delete pv pv-manual-1`.

↑ back to top



STEP 1 · Zero PVs — just a claim

```
$ kubectl get storageclass
NAME          PROVISIONER          RECLAIMPOLICY  BINDINGMODE
standard      k8s.io/minikube-hostpath Delete          Immediate # default ✓
$ kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: dyn-claim } # no className → default class (CH 07)
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 1Gi } }
EOF
$ kubectl get pvc dyn-claim # Bound — and NOBODY wrote a PV:
$ kubectl get pv | grep dyn-claim # pvc-xxxx... auto-created by the provisioner!
```

STEP 2 · Feel the Delete policy bite

```
$ kubectl patch pv <auto-pv-name> \
  -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}' # arm the trap first
$ kubectl delete pvc dyn-claim
$ kubectl get pv <auto-pv-name>
STATUS
Released # Retain: data held hostage (CH 06) — admin must clear claimRef or delete
# WITHOUT the patch: PV + backing storage would be GONE. Delete is the default.
```



Proved: dynamic provisioning needs **zero PV authorship** — and the default Delete policy is one `delete pvc` away from data annihilation unless you Retain first.

Teardown: `kubectl delete pv <auto-pv-name>`

↑ back to top



STEP 1 · Headless service + StatefulSet with a claim template

```
$ kubectl apply -f - <<EOF
apiVersion: v1
kind: Service
metadata: { name: web }
spec: { clusterIP: None, selector: { app: webs }, ports: [{ port: 80 }] }
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: web }
spec:
  serviceName: web
  replicas: 2
  selector: { matchLabels: { app: webs } }
  template:
    metadata: { labels: { app: webs } }
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        volumeMounts: [{ name: data, mountPath: /usr/share/nginx/html }]
  volumeClaimTemplates:
  - metadata: { name: data }
    spec: { accessModes: [ReadWriteOnce],
      resources: { requests: { storage: 100Mi } } }
EOF
$ kubectl get pods -w # web-0 Running → THEN web-1 (ordered! CH 09)
```

ID IDENTITY + STORAGE

```
$ kubectl get pvc
NAME          STATUS  CAPACITY  ← per replica!
data-web-0    Bound   100Mi
data-web-1    Bound   100Mi
$ kubectl run d --rm -it --image=busybox:1.36 \
  --restart=Never -- wget -qO- web-0.web
# per-pod DNS via the HEADLESS svc (CH 09)
```

WRITE PER-REPLICA DATA

```
$ kubectl exec web-0 -- sh -c \
  'echo i-am-zero > /usr/share/nginx/html/i.txt'
$ kubectl exec web-1 -- sh -c \
  'echo i-am-one > /usr/share/nginx/html/i.txt'
# each wrote to ITS OWN disk
```



STEP 2 · Kill web-0 — resurrection, not reincarnation

```
$ kubectl delete pod web-0
$ kubectl get pod web-0 # SAME NAME, back after reschedule
$ kubectl exec web-0 -- cat /usr/share/nginx/html/i.txt
i-am-zero # ← its OWN data survived: data-web-0 reattached
```

STEP 3 · Scale down — the claims stay behind

```
$ kubectl scale sts web --replicas=1
$ kubectl get pvc # data-web-1 STILL BOUND — retained by design (CH 10)
```

Teardown: `kubectl delete sts web && kubectl delete svc web && kubectl delete pvc data-web-0 data-web-1` — the PVCs **never** go with the STS.

[↑ back to top](#)



One claim, three disasters. Which data survives? Run each round and predict *before* looking:

SETUP · Claim + writer (reuse Lab 1's PV or Lab 2's dynamic claim)

```
$ kubectl run g1 --image=busybox:1.36 --restart=Never -- \
  sh -c 'echo round-one > /data/state.txt; sleep 9999'
$ kubectl patch pod g1 --type=merge -p \
  '{"spec":{"volumes":[{"name":"d","persistentVolumeClaim":{"claimName":"surv"}}]}}' 2>/dev/null \
  || kubectl run g1 --image=busybox:1.36 --restart=Never \
  --overrides='{"spec":{"containers":[{"name":"g","image":"busybox:1.36",
  "command":["sh","-c","echo round-one > /data/state.txt; sleep 9999"],
  "volumeMounts":[{"name":"d","mountPath":"/data"}]}],
  "volumes":[{"name":"d","persistentVolumeClaim":{"claimName":"surv"}}]}}'
```

Round	Disaster	Prediction	Verdict
1	<code>kubectl delete pod g1</code>	data...?	SURVIVES — it lives in the PV, not the Pod (Lab 3 proved the pattern)
2	<code>kubectl delete pvc surv</code> — while a Pod uses it	instant gone?	Terminating... hangs — the pvc-protection finalizer (CH 06). Delete the Pod first, then it proceeds
3	PVC gone + PV policy...	data...?	Retain → Released , bytes intact on the node (<code>minikube ssh ls /data/...</code>) · Delete → erased with the disk

```
# the one-liner verdict after every round:
$ kubectl run chk --rm -it --image=busybox:1.36 --restart=Never \
  --overrides='{"spec":{"containers":[{"name":"c","image":"busybox:1.36",
  "command":["cat","/data/state.txt"],
  "volumeMounts":[{"name":"d","mountPath":"/data"}]}],
  "volumes":[{"name":"d","persistentVolumeClaim":{"claimName":"surv"}}]}}'
```



The gauntlet's law: data survives **Pod death** (always, once it's on a PVC), survives **slowly** through PVC death (finalizer + policy), and survives **PV death only under Retain**. Every storage SRE story is one of these three rounds.

Teardown: `kubectl delete pod g1 chk --ignore-not-found; kubectl delete pvc surv; kubectl delete pv pv-manual-1 --ignore-not-found`

↑ back to top



Q1. A container crashes and is restarted inside its Pod. Data written to an `emptyDir` volume...

- A.** survives — `emptyDir` lives with the Pod **B.** is lost with the container layer **C.** moves to another node **D.** becomes read-only

Q2. Which object is **cluster-scoped**?

- A.** `PersistentVolumeClaim` **B.** `StatefulSet` **C.** `PersistentVolume` **D.** `Service`

Q3. `ReadWriteOnce` actually restricts mounting to...

- A.** one Pod **B.** one NODE (any number of its Pods may share) **C.** one container **D.** one namespace

Q4. Which is **NOT** a criterion the PV controller uses to match a claim?

- A.** capacity $\geq$ request **B.** access modes $\supseteq$ requested **C.** `storageClassName` equality **D.** the claiming Pod's name

Q5. Retain policy, PVC deleted. The PV...

- A.** enters **Released** — data kept until an admin intervenes **B.** is deleted with its backing disk **C.** immediately becomes Available for reuse **D.** enters Failed

Q6. A PVC omits `storageClassName`; a default class exists. Result?

- A.** Pending until a manual PV matches **B.** validation error **C.** dynamically provisioned from the default class **D.** binds only PVs labeled default

Q7. `volumeBindingMode: WaitForFirstConsumer` exists to...

- A.** slow down provisioning for safety **B.** delay binding until a consuming Pod is scheduled — so zoned disks are born in the right zone **C.** require admin approval **D.** apply only to RWX volumes

Q8. A `StatefulSet` scales 3 → 2 replicas. The removed replica's PVC...

- A.** is deleted immediately **B.** is renamed **C.** is transferred to a surviving replica **D.** is retained by default — delete it manually if truly unwanted

Q9. `volumeClaimTemplates` means...

- A.** all replicas share one claim **B.** admins must pre-create one PVC per replica **C.** each replica gets its own PVC stamped from the template, named after its Pod **D.** claims are optional

Q10. CSI's core achievement is...

- A.** storage drivers hard-coded in the k8s tree **B.** vendors ship storage drivers as out-of-tree containers against a standard interface **C.** eliminating PVs entirely **D.** automatic encryption of all volumes

[↑ back to top](#)



Q	Ans	Why — one line
1	A	emptyDir outlives containers, dies with the Pod — the whole three-lives ladder (CH 01–02)
2	C	PV = supply = cluster-wide; PVC/StatefulSet/Service are namespaced — Book 01 · CH 11 (CH 03)
3	B	The unit is NODES — same-node Pods may share one RWO volume (CH 05)
4	D	Pod names never enter matching: capacity, modes, class, selector only (CH 04)
5	A	Retain = Released, data hostage; clearing claimRef re-opens it (CH 06)
6	C	Omitted field + default class = dynamic provisioning; "" would opt out (CH 07)
7	B	Wait for the consumer → disk born in the Pod's zone; fixes node-affinity conflict wedges (CH 08)
8	D	Data-safety by design: claims outlive scaled-down replicas (CH 10, Lab 3)
9	C	Template = cookie cutter: data-<sts>-<ordinal> per replica (CH 10)
10	B	Standard interface + installable driver containers; in-tree frozen (CH 11)



9–10

Ready for Book 06



6–8

Re-read missed chapters'  boxes

<6

Re-run Labs 1–4, then retake



Missed Q3? The RWO-is-a-node lie is the most common storage misconception in interviews — drill it. Missed Q8 or Q9? StatefulSet storage behavior (claims outlive scale-down) is exactly what "stateful" *means*.

[↑ back to top](#)



Volume — a Pod-level directory mounted into containers; lifespan set by its type (emptyDir, hostPath, configMap, PVC...).

emptyDir — scratch volume sharing the Pod's life; survives container restarts, dies with the Pod; optional sizeLimit.

hostPath — node-local path; couples Pods to nodes and can expose the whole filesystem — daemon-only territory.

PersistentVolume (PV) — cluster-scoped piece of real storage with capacity, accessModes, reclaimPolicy; the supply.

PersistentVolumeClaim (PVC) — namespaced request (capacity + modes + class + selector); the only thing Pods mount.

Binding — 1:1 PV→PVC match: capacity $\geq$, modes $\supseteq$, class $=$, selector match; smallest fitting PV wins.

Access modes — RWO (one NODE!), ROX, RWX (many nodes, shared filesystem), RWOP (one Pod, GA 1.29).

Multi-Attach error — a block volume refusing simultaneous attach to two nodes; the RWO-wedge signature.

PV phases — Available → Bound → Released (Retain holds here) → Failed; Released never auto-rebinds.

Reclaim policy — Retain (keep data, admin intervenes) / Delete (PV + backing storage destroyed; dynamic default) / Recycle (removed).

pvc-protection finalizer — keeps a PVC Terminating while a Pod still uses it.

StorageClass — named provisioning recipe: provisioner + parameters + reclaimPolicy + bindingMode + expansion flag.

Dynamic provisioning — claim names a class → provisioner builds storage + PV automatically; default class serves omitted fields; "" opts out.

volumeBindingMode — Immediate (bind now) vs WaitForFirstConsumer (bind when a Pod schedules — zone-correct).

allowVolumeExpansion — permits PVC growth (never shrink); filesystem resize online.

StatefulSet — workload controller giving replicas stable names (web-0...), per-Pod DNS via a headless Service, and ordered lifecycle.

Headless Service — clusterIP: None; DNS resolves each Pod individually — how followers find the primary by name.

volumeClaimTemplates — per-replica claim cookie-cutter: data-<sts>-<ordinal>; born on scale-up, retained on scale-down and on STS deletion.

CSI — Container Storage Interface: standard gRPC contract; drivers ship as controller (Deployment) + node (DaemonSet) plugins, out-of-tree.

volumeMode — Filesystem (formatted dir, default) vs Block (raw device passed to the Pod).

[↑ back to top](#)



STORAGE — COMMANDS

```
$ kubectl get pv,pvc,sc
$ kubectl get pvc # Pending? check 4 match rules
$ kubectl describe pv X # node affinity = its zone
$ kubectl patch pv X -p \
  '{"spec":
{"persistentVolumeReclaimPolicy":"Retain"}}'
$ kubectl patch pv X -p '{"spec":
{"claimRef":null}}' # Released→Available
$ kubectl patch pvc P -p \
  '{"spec":{"resources":{"requests":
{"storage":"10Gi"}}}}' # grow only
```

PRODUCTION STORAGECLASS

```
provisioner: ebs.csi.aws.com
parameters: {type: gp3}
reclaimPolicy: Retain # not Delete!
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
# default class annotation:
storageclass.kubernetes.io/is-default-class:
"true"
```

STATEFULSET STORAGE SKELETON

```
serviceName: db # headless: clusterIP None
volumeClaimTemplates:
- metadata: {name: data}
  spec: {accessModes: [ReadWriteOnce],
        resources: {requests: {storage: 10Gi}}}
# → data-db-0, data-db-1 ...
# scale-up: born · scale-down: RETAINED
# delete STS: claims survive — manual cleanup
```

TRIAGE TABLE

Symptom	Meaning
PVC Pending	class/capacity/mode/selector mismatch
Pod Pending + node affinity conflict	disk in wrong zone → WaitForFirstConsumer
Multi-Attach error	RWO volume wanted on 2 nodes
PVC Terminating forever	pvc-protection: Pod still using it
PV Released	Retain after claim death — admin's move

FACTS WORTH A JOB OFFER

Three lives of data: container layer (dies with container) → emptyDir (dies with Pod) → PV (outlives everything)

PV = cluster-scoped supply · PVC = namespaced demand · Pods mount the claim, never the volume

RWO = one NODE, not one Pod — same-node Pods share it; RWX needs shared backing (NFS/EFS/Ceph)

Retain keeps Released data hostage; Delete (dynamic default) erases PV + disk — patch before you delete

WaitForFirstConsumer = disks born in the Pod's zone; storage grows but never shrinks

StatefulSet claims: one per replica, named after the Pod, kept on scale-down and STS deletion

CSI = the universal plug; drivers are containers (Deployment + DaemonSet), out-of-tree, independent releases

[↑ back to top](#)



Book 01 built the machine, 02 dressed it, 03 connected it, 04 placed it — Book 05 gave it **memory that survives death**.

STORAGE — fully mapped:

└ Ladder	container layer → emptyDir → PV (three lives of data)
└ PV / PVC	cluster supply ⇔ namespaced demand · 4-rule binding
└ Access modes	RWO = one NODE · RWX needs shared backing
└ Reclaim	Available→Bound→Released→{Retain Delete}
└ StorageClass	dynamic recipes · WaitForFirstConsumer · grow-only
└ StatefulSets	identity + per-replica claims that outlive scale-down
└ CSI	drivers as containers — the universal plug

✓ You can now...

Trace any byte of cluster data through its three lives and say exactly which disaster erases it

Bind claims by hand AND provision dynamically — and explain the four matching rules when a PVC goes Pending

Decode RWO/RWX correctly, diagnose Multi-Attach wedges, and pick the right backing per workload

Write production StorageClasses: Retain policy, zone-correct binding, grow-only expansion

Run a StatefulSet knowing precisely which disks belong to which replica — and what survives scale-down

Read any `provisioner: ...csi...` line and know the whole machinery behind it



Where to next — Book 06 · Higher-Level Workloads: you've met Deployments and tasted StatefulSets — Book 06 completes the zoo: **DaemonSets** (one Pod per node — you've seen them skip drains!), **Jobs & CronJobs** (run-to-completion), **StatefulSets in full** (rolling updates, partition canaries), and the autoscaling trio **HPA/VPA/Cluster Autoscaler**. Your workloads can persist — now they learn to run themselves.

— End of Book 05 ·  —